

Journal de bord & Rapport technique — CAT Mobile Application

Module : Développement d'applications mobiles

Projet : Nikah Planner — Application de planification de mariage

Étudiant : Hadidja Aliani

ID étudiant : BIT/2023/62116

Compte GitHub : [moinalia](#)

Dépôt source : <https://github.com/moinalia/nikah-planner>

Date de soumission : Juin 2026

Table des matières

1. [Vue d'ensemble du projet](#)
 2. [Couverture des fonctionnalités CAT](#)
 3. [Modules implémentés](#)
 4. [Grille de notation \(auto-évaluation\)](#)
 5. [Architecture technique](#)
 6. [Stockage local et récupération des données](#)
 7. [Réseau et consommation d'API](#)
 8. [Gestion des erreurs](#)
 9. [Phases de développement \(logbook\)](#)
 10. [Instructions de test et démonstration](#)
 11. [Captures d'écran requises](#)
 12. [Questions de révision](#)
 13. [Références](#)
 14. [Annexes](#)
-

1. Vue d'ensemble du projet

1.1 Contexte

Nikah Planner est une application mobile Android permettant aux couples de planifier leur mariage : budget, invités, planning des cérémonies, prestataires et notifications. Le projet poursuit le prototype UI développé en début de semestre et l'intègre dans une application Flutter complète.

1.2 Objectifs CAT

Développer une application mobile complète intégrant les concepts des semaines 1 à 5 :

Exigence CAT	Implémentation Nikah Planner
User Interface Design	13 écrans Material Design, thème or/sauge
Navigation	<code>go_router</code> + barre de navigation à 5 onglets
Event Handling	Boutons, formulaires, filtres, <code>onTap</code> , <code>onChanged</code>
Local Data Storage	Données locales (<code>mock_data.dart</code>) + <code>SharedPreferences</code>
Data Retrieval	Lecture des données pour chaque écran
Networking	Firebase Auth + Firestore (cloud)
API Consumption	REST via SDK Firebase
Error Handling	Messages d'erreur login/register, validation formulaires

1.3 Technologies

Couche	Technologie
Mobile	Flutter 3.x / Dart
Navigation	<code>go_router</code>
État	Riverpod
Stockage local	<code>SharedPreferences</code> + données embarquées
Base cloud	Firebase Auth + Cloud Firestore
Prototype web	React + Vite (démonstration navigateur)

Couche	Technologie
Versionnement	Git / GitHub

2. Couverture des fonctionnalités CAT

2.1 User Interface Design (5 marks)

Implémentation :

- Splash screen animé (`lib/features/splash/splash_screen.dart`)
- Thème cohérent : couleurs or (#B8956A), sauge (#8FB5B0), polices Playfair Display + DM Sans
- Composants réutilisables : `WeddingCard` , `PrimaryButton` , `WeddingInputField`
- 13 écrans responsive avec `CustomScrollView` , graphiques (`fl_chart`)

Fichiers clés : `lib/core/theme/` , `lib/core/widgets/wedding_widgets.dart`

2.2 Navigation (4 marks)

Implémentation :

- `GoRouter` avec routes nommées (`lib/router/app_router.dart`)
- Navigation par onglets : Home, Budget, Guests, Gallery, More (`MainShell`)
- Navigation empilée vers sous-pages : Schedule, Vendors, Profile, Settings
- Redirection : Splash → Login → Dashboard

Exemple de routes :

```
/splash → /login → /register → /home
/home, /budget, /guests, /gallery, /more
/schedule, /vendors, /profile, /settings, /notifications
```

2.3 Event Handling (inclus dans UI/Navigation)

Exemples d'événements gérés :

- Tap sur boutons Login / Register
- Saisie texte dans `TextEditingController`
- Filtres RSVP et côté mariée/marié (Guest List)
- Expansion des événements dans Schedule (`setState`)

- Toggle notifications dans Settings
- Navigation `onTap` vers sous-écrans

Fichiers : `login_screen.dart` , `register_screen.dart` , `guest_list_screen.dart` , `schedule_screen.dart`

2.4 Local Data Storage (5 marks)

Implémentation :

Méthode	Rôle	Fichier
Données embarquées	Budget, invités, événements (mode démo)	<code>lib/core/data/mock_data.dart</code>
SharedPreferences	Comptes utilisateurs et profils mariage (JSON)	<code>lib/services/local_wedding_store.dart</code>
Session mémoire	Données du couple connecté	<code>lib/services/wedding_session.dart</code>

Structure stockage local (SharedPreferences) :

```

nikah_local_users    → { userId: { email, password, name, weddingId } }
nikah_local_weddings → { weddingId: { profil JSON complet } }
nikah_email_index    → { email: userId }
```

Différence temporaire vs permanent : voir section 12.1

2.5 Data Retrieval (4 marks)

Implémentation :

- `MockData` expose getters lus par tous les écrans (budget, invités, events...)
- `WeddingRepository.loadForUser()` récupère le profil depuis SharedPreferences ou Firestore
- `WeddingSession` fournit les données actives après connexion
- Filtres invités : recherche + RSVP + côté mariée/marié

Flux :

```

Login → loadForUser(userId) → WeddingSession.activate(profile) → MockData lit la session → UI affiche
```

2.6 Networking (5 marks)

Implémentation :

- **Firestore Authentication** : création compte, connexion email/mot de passe
- **Cloud Firestore** : collections `users/` , `weddings/` , sous-collections `guests/`
- SDK Firebase = client HTTP asynchrone vers API Google Cloud

Fichiers : `lib/services/auth_service.dart` , `lib/services/firestore_service.dart`

Mode démo (sans réseau) : l'app fonctionne hors ligne avec stockage local si Firebase n'est pas configuré.

2.7 API Consumption (5 marks)

API utilisée : Firebase REST/SDK

Opération	Type	Service
Inscription	POST (createUser)	Firebase Auth
Connexion	POST (signIn)	Firebase Auth
Créer mariage	POST (set document)	Firestore
Lire profil	GET (get document)	Firestore
Stream invités	GET (snapshots)	Firestore

Format données : JSON (Firestore documents, sérialisation `WeddingProfile.toJson()`)

2.8 Error Handling (3 marks)

Implémentation :

Situation	Gestion
Champs vides login	Message : "Please enter email and password"
Mot de passe < 8 caractères	Message à l'inscription
Email déjà enregistré	"This email is already registered"
Compte introuvable	"Account not found. Please register first"

Situation	Gestion
Mauvais mot de passe	"Incorrect password"
Erreur Firebase	<code>FirebaseAuthException</code> → message affiché
Firebase non configuré	Fallback mode démo automatique

Fichier : `lib/services/auth_service.dart` → classe `AuthResult`

3. Modules implémentés

Correspondance avec les modules suggérés du CAT :

Module suggéré	Module Nikah Planner	Fichier
1. Login Screen	Écran de connexion	<code>lib/features/auth/login_screen.dart</code>
2. Dashboard	Accueil (compte à rebours, tâches, stats)	<code>lib/features/home/home_screen.dart</code>
3. Student Registration	Inscription couple / utilisateur	<code>lib/features/auth/register_screen.dart</code>
4. Local Database	SharedPreferences + mock data	<code>local_wedding_store.dart</code> , <code>mock_data.dart</code>
5. API Integration	Firebase Auth + Firestore	<code>auth_service.dart</code> , <code>firestore_service.dart</code>
6. Reports Screen	Rapport budget (graphique + transactions)	<code>lib/features/budget/budget_screen.dart</code>

Écrans supplémentaires : Guests, Gallery, Schedule, Vendors, Profile, Invitations, Notifications, Settings.

4. Grille de notation (auto-évaluation)

Critère	Marks	Preuve dans le projet
User Interface Design	/5	13 écrans, thème cohérent, composants réutilisables

Critère	Marks	Preuve dans le projet
Navigation	/4	GoRouter, 5 onglets, sous-pages
Local Storage	/5	SharedPreferences + mock_data
Data Retrieval	/4	MockData, WeddingRepository, filtres
Networking/API	/5	Firebase Auth + Firestore
Error Handling	/3	AuthResult, messages UI
Code Quality	/2	Structure features/, services/, lints Flutter
Documentation	/2	README, ce journal, SOUMISSION_PROF.md
Total	/30	

5. Architecture technique



Pattern : Repository pattern — `WeddingRepository` abstrait la source (local ou cloud).

6. Stockage local et récupération des données

6.1 Données embarquées (mode démo professeur)

Fichier `mock_data.dart` — données d'exemple Siti & Ahmad pour tester sans compte :

- 8 catégories budget, 8 invités, 6 événements, 7 prestataires

6.2 Stockage persistant (SharedPreferences)

Chaque utilisateur inscrit obtient son propre espace :

- Inscription → `WeddingDataFactory.createNew()` → sauvegarde JSON
- Connexion → `loadForUser()` → affichage données personnelles

6.3 Récupération

```
// Exemple simplifié
final profile = await WeddingRepository.instance.loadForUser(userId);
WeddingSession.activate(profile);
// Les écrans lisent MockData qui pointe vers WeddingSession
```

7. Réseau et consommation d'API

7.1 Architecture client-serveur

```
[App Flutter] ↔ [Firebase Auth API]    (authentification)
[App Flutter] ↔ [Cloud Firestore API]   (données mariage)
```

7.2 Collections Firestore

```
users/{userId}          → email, name, role, weddingId
weddings/{weddingId}    → brideName, groomName, weddingDate, budget...
weddings/{id}/guests/   → liste invités
```

7.3 Traitement asynchrone

Toutes les opérations réseau utilisent `async/await` :

- `signIn()`, `register()`, `createWeddingProfile()`, `loadForUser()`
-

8. Gestion des erreurs

```
// Pattern AuthResult utilisé dans toute l'app
final result = await authService.signIn(email: email, password: password);
if (result.isSuccess) {
  context.go('/home');
} else {
  setState(() => _error = result.error); // Affichage à l'utilisateur
}
```

Principes :

- Validation côté client avant appel API
- Capture `FirebaseAuthException` avec message lisible
- Fallback gracieux si Firebase indisponible (mode démo)

9. Phases de développement (logbook)

Semaine	Date	Activité	Statut
1	[Date]	Analyse besoins, maquette UI React	✓
2	[Date]	Création projet Flutter, thème, Splash	✓
3	[Date]	Login, Register, navigation GoRouter	✓
4	[Date]	Dashboard, Budget, Guests, Schedule	✓
5	[Date]	Stockage local SharedPreferences	✓
5	[Date]	Intégration Firebase (Auth + Firestore)	✓
6	[Date]	Tests Android, build APK, publication GitHub	✓
6	[Date]	Documentation et journal de bord	✓

10. Instructions de test et démonstration

10.1 Pour le professeur — Application mobile

```
git clone https://github.com/moinaalia/nikah-planner.git
cd nikah-planner/nikah_planner
flutter pub get
flutter run
```

Démonstration recommandée :

1. Splash screen → Login
2. Connexion mode démo (n'importe quel email + mot de passe)
3. Dashboard : compte à rebours, tâches, stats budget
4. Budget : graphique camembert + transactions (Reports)
5. Invités : filtres RSVP
6. Schedule : timeline des événements
7. Settings → Sign Out

10.2 Prototype web (navigateur)

```
cd nikah-planner
npm install
npm run dev
# Ouvrir http://localhost:5173
```

10.3 Avec compte personnel (inscription)

1. Register → remplir formulaire → Create Account
2. Chaque email = espace privé séparé
3. Sign Out → créer 2e compte → montrer données différentes

11. Captures d'écran requises

À joindre au rapport (PDF ou dossier `screenshots/`) :

#	Écran	Fichier suggéré
1	Splash	01_splash.png
2	Login	02_login.png

#	Écran	Fichier suggéré
3	Register	03_register.png
4	Dashboard (Home)	04_dashboard.png
5	Budget / Reports	05_budget.png
6	Guest List + filtres	06_guests.png
7	Schedule	07_schedule.png
8	Settings + profil	08_settings.png
9	Message d'erreur login	09_error.png

Comment capturer : émulateur Android → bouton caméra, ou `flutter run` sur téléphone → screenshot.

12. Questions de révision

12.1 Différencier stockage temporaire et permanent

Temporaire	Permanent
RAM / session mémoire (<code>WeddingSession</code>)	SharedPreferences sur disque
Perdu à la fermeture de l'app	Persiste après redémarrage
Variables <code>setState</code>	Fichier JSON dans SharedPreferences
Exemple : filtre invité actif	Exemple : compte utilisateur sauvegardé

12.2 Expliquer SQLite

SQLite est une base de données relationnelle légère embarquée dans l'appareil mobile. Elle stocke des données dans des tables avec SQL (SELECT, INSERT, UPDATE). Alternative à SharedPreferences pour des données structurées complexes. *Nikah Planner utilise SharedPreferences (format clé-valeur JSON) ; SQLite peut être ajouté pour les invités à grande échelle.*

12.3 Définir API

Une **API** (Application Programming Interface) est une interface permettant à deux programmes de communiquer. En mobile, l'app (client) envoie des requêtes HTTP à un serveur (ex. Firebase) et reçoit des réponses JSON.

12.4 Expliquer le traitement asynchrone

Le traitement **asynchrone** permet à l'app de continuer à fonctionner pendant une opération longue (réseau, disque). En Dart : `async/await` et `Future`. Exemple : `await authService.signIn()` n'bloque pas l'interface.

12.5 Comparer GET et POST

GET	POST
Récupérer des données	Envoyer / créer des données
Paramètres dans l'URL	Données dans le corps
Exemple : lire profil Firestore	Exemple : créer compte Firebase Auth
Idempotent	Peut modifier le serveur

12.6 Architecture client-serveur

Le **client** (app mobile Flutter) envoie des requêtes au **serveur** (Firebase Cloud). Le serveur traite, accède à la base de données et renvoie une réponse. L'app n'accède jamais directement à la base serveur sans passer par l'API.

12.7 Expliquer JSON

JSON (JavaScript Object Notation) est un format texte pour échanger des données structurées : `{ "brideName": "Siti", "budget": 15000 }`. Utilisé par Firestore et SharedPreferences dans Nikah Planner.

12.8 Concevoir une app intégrant stockage et réseau

Nikah Planner illustre ce design :

1. **Inscription** → POST Firebase Auth + sauvegarde locale
 2. **Lecture** → GET Firestore ou SharedPreferences
 3. **Affichage** → UI Flutter lit `MockData` / `WeddingSession`
 4. **Hors ligne** → fallback stockage local si pas de réseau
-

13. Références

1. Android Developer Documentation — <https://developer.android.com>
 2. Flutter Documentation — <https://docs.flutter.dev>
 3. Firebase Documentation — <https://firebase.google.com/docs>
 4. SQLite Documentation — <https://www.sqlite.org/docs.html>
 5. REST API Design — <https://restfulapi.net>
 6. SharedPreferences (Flutter) — package `shared_preferences`
 7. Dépôt projet — <https://github.com/moinaalia/nikah-planner>
-

14. Annexes

A. Liens de soumission

Élément	Lien
Code source	https://github.com/moinaalia/nikah-planner
Profil GitHub	https://github.com/moinaalia
README	https://github.com/moinaalia/nikah-planner#readme

B. Checklist soumission CAT

- ☐ Code source sur GitHub
- ☒ Captures d'écran (section 11)
- ☐ Ce rapport technique (PDF)
- ☐ Démonstration live (émulateur ou téléphone)

C. Structure du dépôt

```
nikah-planner/  
├─ nikah_planner/      ← App Flutter (CAT principal)  
├─ src/                ← Prototype React  
├─ README.md  
├─ SOUMISSION_PROF.md  
└─ JOURNAL_DE_BORD.md ← Ce document
```

Rapport rédigé conformément aux instructions CAT — Mobile Application Development.